

Data Manipulation with Hotel Booking Dataset

Keman Xiang

Background

In this notebook, we focus on manipulating and changing real data in R. we utilize functions to manipulate our data, use statistical summaries to explore our data, and gain initial insights for our stakeholders.

The Scenario

In this scenario, we are junior data analysts working for a hotel booking company. we have been asked to clean a .csv file that was created after querying a database to combine two different tables from different hotels. we have already performed some basic cleaning functions on this data; this notebook will focus on using functions to conduct basic data manipulation.

Step 1: Load packages

Start by installing the required packages.

```
#install.packages("tidyverse")
#install.packages("skimr")
#install.packages("janitor")
```

load packages:

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.4
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    3.4.4      ✓ tibble     3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr      1.3.0
## ✓ purrr      1.0.2
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(skimr)
library(janitor)
```

```
##  
## Attaching package: 'janitor'  
##  
## The following objects are masked from 'package:stats':  
##  
##   chisq.test, fisher.test
```

Step 2: Import data

The data in this example is originally from the article Hotel Booking Demand Datasets

(<https://www.sciencedirect.com/science/article/pii/S2352340918315191>

(<https://www.sciencedirect.com/science/article/pii/S2352340918315191>)), written by Nuno Antonio, Ana Almeida, and Luis Nunes for Data in Brief, Volume 22, February 2019.

The data was downloaded and cleaned by Thomas Mock and Antoine Bichat for #TidyTuesday during the week of February 11th, 2020 (<https://github.com/rfordatascience/tidytuesday/blob/master/data/2020/2020-02-11/readme.md> (<https://github.com/rfordatascience/tidytuesday/blob/master/data/2020/2020-02-11/readme.md>)).

We can learn more about the dataset here: <https://www.kaggle.com/jessemostipak/hotel-booking-demand> (<https://www.kaggle.com/jessemostipak/hotel-booking-demand>)

Import data from a .csv in the project folder called “hotel_bookings.csv” and save it as a data frame called hotel_bookings :

```
hotel_bookings <- read_csv("hotel_bookings.csv")
```

```
## Rows: 119390 Columns: 32  
## — Column specification —————  
## Delimiter: ","  
## chr  (13): hotel, arrival_date_month, meal, country, market_segment, distrib...  
## dbl  (18): is_canceled, lead_time, arrival_date_year, arrival_date_week_numb...  
## date  (1): reservation_status_date  
##  
## i Use `spec()` to retrieve the full column specification for this data.  
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Step 3: Getting to know our data

Like we have been doing in other notebooks, use the `head()` function to preview the columns and the first several rows of data

```
head(hotel_bookings)
```

```
## # A tibble: 6 × 32
##   hotel          is_canceled lead_time arrival_date_year arrival_date_month
##   <chr>          <dbl>      <dbl>          <dbl> <chr>
## 1 Resort Hotel      0        342            2015 July
## 2 Resort Hotel      0        737            2015 July
## 3 Resort Hotel      0         7            2015 July
## 4 Resort Hotel      0         13            2015 July
## 5 Resort Hotel      0         14            2015 July
## 6 Resort Hotel      0         14            2015 July
## # i 27 more variables: arrival_date_week_number <dbl>,
## #   arrival_date_day_of_month <dbl>, stays_in_weekend_nights <dbl>,
## #   stays_in_week_nights <dbl>, adults <dbl>, children <dbl>, babies <dbl>,
## #   meal <chr>, country <chr>, market_segment <chr>,
## #   distribution_channel <chr>, is_repeated_guest <dbl>,
## #   previous_cancellations <dbl>, previous_bookings_not_canceled <dbl>,
## #   reserved_room_type <chr>, assigned_room_type <chr>, ...
```

Data Analysis

1. How many columns are in this data set? There are 32 columns in this data set. The `head()` function returns a preview of the data set, including the first six rows of data and as many columns as will fit on the screen. At the bottom left of the table, it states that it is previewing 1-4 of 32 columns.
2. The 'arrival_date_month' variable is chr or character type data.
Underneath the column name in the preview table, there is a description of the data type for each column.

In addition to `head()` we can also use the `str()` and `glimpse()` functions to get summaries of each column in our data arranged horizontally.

```
str(hotel_bookings)
```

```

## spc_tbl_ [119,390 × 32] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ hotel : chr [1:119390] "Resort Hotel" "Resort Hotel" "Reso
rt Hotel" "Resort Hotel" ...
## $ is_canceled : num [1:119390] 0 0 0 0 0 0 0 0 1 1 ...
## $ lead_time : num [1:119390] 342 737 7 13 14 14 0 9 85 75 ...
## $ arrival_date_year : num [1:119390] 2015 2015 2015 2015 2015 ...
## $ arrival_date_month : chr [1:119390] "July" "July" "July" "July" ...
## $ arrival_date_week_number : num [1:119390] 27 27 27 27 27 27 27 27 27 ...
## $ arrival_date_day_of_month : num [1:119390] 1 1 1 1 1 1 1 1 1 ...
## $ stays_in_weekend_nights : num [1:119390] 0 0 0 0 0 0 0 0 0 ...
## $ stays_in_week_nights : num [1:119390] 0 0 1 1 2 2 2 2 3 3 ...
## $ adults : num [1:119390] 2 2 1 1 2 2 2 2 2 2 ...
## $ children : num [1:119390] 0 0 0 0 0 0 0 0 0 0 ...
## $ babies : num [1:119390] 0 0 0 0 0 0 0 0 0 0 ...
## $ meal : chr [1:119390] "BB" "BB" "BB" "BB" ...
## $ country : chr [1:119390] "PRT" "PRT" "GBR" "GBR" ...
## $ market_segment : chr [1:119390] "Direct" "Direct" "Direct" "Corpora
te" ...
## $ distribution_channel : chr [1:119390] "Direct" "Direct" "Direct" "Corpora
te" ...
## $ is_repeated_guest : num [1:119390] 0 0 0 0 0 0 0 0 0 0 ...
## $ previous_cancellations : num [1:119390] 0 0 0 0 0 0 0 0 0 0 ...
## $ previous_bookings_not_canceled : num [1:119390] 0 0 0 0 0 0 0 0 0 0 ...
## $ reserved_room_type : chr [1:119390] "C" "C" "A" "A" ...
## $ assigned_room_type : chr [1:119390] "C" "C" "C" "A" ...
## $ booking_changes : num [1:119390] 3 4 0 0 0 0 0 0 0 0 ...
## $ deposit_type : chr [1:119390] "No Deposit" "No Deposit" "No Depos
it" "No Deposit" ...
## $ agent : chr [1:119390] "NULL" "NULL" "NULL" "304" ...
## $ company : chr [1:119390] "NULL" "NULL" "NULL" "NULL" ...
## $ days_in_waiting_list : num [1:119390] 0 0 0 0 0 0 0 0 0 0 ...
## $ customer_type : chr [1:119390] "Transient" "Transient" "Transient"
"Transient" ...
## $ adr : num [1:119390] 0 0 75 75 98 ...
## $ required_car_parking_spaces : num [1:119390] 0 0 0 0 0 0 0 0 0 0 ...
## $ total_of_special_requests : num [1:119390] 0 0 0 0 1 1 0 1 1 0 ...
## $ reservation_status : chr [1:119390] "Check-Out" "Check-Out" "Check-Out"
"Check-Out" ...
## $ reservation_status_date : Date[1:119390], format: "2015-07-01" "2015-07-01"
...
## - attr(*, "spec")=
## .. cols(
## .. hotel = col_character(),
## .. is_canceled = col_double(),
## .. lead_time = col_double(),
## .. arrival_date_year = col_double(),
## .. arrival_date_month = col_character(),
## .. arrival_date_week_number = col_double(),
## .. arrival_date_day_of_month = col_double(),
## .. stays_in_weekend_nights = col_double(),
## .. stays_in_week_nights = col_double(),
## .. adults = col_double(),

```

```
## .. children = col_double(),  
## .. babies = col_double(),  
## .. meal = col_character(),  
## .. country = col_character(),  
## .. market_segment = col_character(),  
## .. distribution_channel = col_character(),  
## .. is_repeated_guest = col_double(),  
## .. previous_cancellations = col_double(),  
## .. previous_bookings_not_canceled = col_double(),  
## .. reserved_room_type = col_character(),  
## .. assigned_room_type = col_character(),  
## .. booking_changes = col_double(),  
## .. deposit_type = col_character(),  
## .. agent = col_character(),  
## .. company = col_character(),  
## .. days_in_waiting_list = col_double(),  
## .. customer_type = col_character(),  
## .. adr = col_double(),  
## .. required_car_parking_spaces = col_double(),  
## .. total_of_special_requests = col_double(),  
## .. reservation_status = col_character(),  
## .. reservation_status_date = col_date(format = "")  
## .. )  
## - attr(*, "problems")=<externalptr>
```

we can see the different column names and some sample values to the right of the colon.

```
glimpse(hotel_bookings)
```

```
## Rows: 119,390
## Columns: 32
## $ hotel <chr> "Resort Hotel", "Resort Hotel", "Resort...
## $ is_canceled <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, ...
## $ lead_time <dbl> 342, 737, 7, 13, 14, 14, 0, 9, 85, 75, ...
## $ arrival_date_year <dbl> 2015, 2015, 2015, 2015, 2015, 2015, 201...
## $ arrival_date_month <chr> "July", "July", "July", "July", "July",...
## $ arrival_date_week_number <dbl> 27, 27, 27, 27, 27, 27, 27, 27, 27, 27,...
## $ arrival_date_day_of_month <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
## $ stays_in_weekend_nights <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ stays_in_week_nights <dbl> 0, 0, 1, 1, 2, 2, 2, 2, 3, 3, 4, 4, 4, ...
## $ adults <dbl> 2, 2, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, ...
## $ children <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ babies <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ meal <chr> "BB", "BB", "BB", "BB", "BB", "BB", "BB", "BB...
## $ country <chr> "PRT", "PRT", "GBR", "GBR", "GBR", "GBR...
## $ market_segment <chr> "Direct", "Direct", "Direct", "Corporat...
## $ distribution_channel <chr> "Direct", "Direct", "Direct", "Corporat...
## $ is_repeated_guest <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ previous_cancellations <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ previous_bookings_not_canceled <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ reserved_room_type <chr> "C", "C", "A", "A", "A", "A", "C", "C",...
## $ assigned_room_type <chr> "C", "C", "C", "A", "A", "A", "C", "C",...
## $ booking_changes <dbl> 3, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ deposit_type <chr> "No Deposit", "No Deposit", "No Deposit...
## $ agent <chr> "NULL", "NULL", "NULL", "304", "240", "...
## $ company <chr> "NULL", "NULL", "NULL", "NULL", "NULL",...
## $ days_in_waiting_list <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ customer_type <chr> "Transient", "Transient", "Transient", ...
## $ adr <dbl> 0.00, 0.00, 75.00, 75.00, 98.00, 98.00,...
## $ required_car_parking_spaces <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ total_of_special_requests <dbl> 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 3, ...
## $ reservation_status <chr> "Check-Out", "Check-Out", "Check-Out", ...
## $ reservation_status_date <date> 2015-07-01, 2015-07-01, 2015-07-02, 20...
```

we can also use `colnames()` to get the names of the columns in our data set.

```
colnames(hotel_bookings)
```

```
## [1] "hotel" "is_canceled"
## [3] "lead_time" "arrival_date_year"
## [5] "arrival_date_month" "arrival_date_week_number"
## [7] "arrival_date_day_of_month" "stays_in_weekend_nights"
## [9] "stays_in_week_nights" "adults"
## [11] "children" "babies"
## [13] "meal" "country"
## [15] "market_segment" "distribution_channel"
## [17] "is_repeated_guest" "previous_cancellations"
## [19] "previous_bookings_not_canceled" "reserved_room_type"
## [21] "assigned_room_type" "booking_changes"
## [23] "deposit_type" "agent"
## [25] "company" "days_in_waiting_list"
## [27] "customer_type" "adr"
## [29] "required_car_parking_spaces" "total_of_special_requests"
## [31] "reservation_status" "reservation_status_date"
```

Manipulating our data

Let's say we want to arrange the data by most lead time to least lead time because we want to focus on bookings that were made far in advance. We decide we want to try using the `arrange()` function:

```
arrange(hotel_bookings, lead_time)
```

```
## # A tibble: 119,390 × 32
##   hotel          is_canceled lead_time arrival_date_year arrival_date_month
##   <chr>          <dbl>      <dbl>          <dbl> <chr>
## 1 Resort Hotel      0          0            2015 July
## 2 Resort Hotel      0          0            2015 July
## 3 Resort Hotel      0          0            2015 July
## 4 Resort Hotel      0          0            2015 July
## 5 Resort Hotel      0          0            2015 July
## 6 Resort Hotel      0          0            2015 July
## 7 Resort Hotel      0          0            2015 July
## 8 Resort Hotel      0          0            2015 July
## 9 Resort Hotel      0          0            2015 July
## 10 Resort Hotel     0          0            2015 July
## # i 119,380 more rows
## # i 27 more variables: arrival_date_week_number <dbl>,
## #   arrival_date_day_of_month <dbl>, stays_in_weekend_nights <dbl>,
## #   stays_in_week_nights <dbl>, adults <dbl>, children <dbl>, babies <dbl>,
## #   meal <chr>, country <chr>, market_segment <chr>,
## #   distribution_channel <chr>, is_repeated_guest <dbl>,
## #   previous_cancellations <dbl>, previous_bookings_not_canceled <dbl>, ...
```

`arrange()` automatically orders by ascending order, and we need to specifically tell it when to order by descending order:

```
arrange(hotel_bookings, desc(lead_time))
```

```
## # A tibble: 119,390 × 32
##   hotel          is_canceled lead_time arrival_date_year arrival_date_month
##   <chr>          <dbl>      <dbl>          <dbl> <chr>
## 1 Resort Hotel      0        737            2015 July
## 2 Resort Hotel      0        709            2016 February
## 3 City Hotel        1        629            2017 March
## 4 City Hotel        1        629            2017 March
## 5 City Hotel        1        629            2017 March
## 6 City Hotel        1        629            2017 March
## 7 City Hotel        1        629            2017 March
## 8 City Hotel        1        629            2017 March
## 9 City Hotel        1        629            2017 March
## 10 City Hotel       1        629            2017 March
## # i 119,380 more rows
## # i 27 more variables: arrival_date_week_number <dbl>,
## #   arrival_date_day_of_month <dbl>, stays_in_weekend_nights <dbl>,
## #   stays_in_week_nights <dbl>, adults <dbl>, children <dbl>, babies <dbl>,
## #   meal <chr>, country <chr>, market_segment <chr>,
## #   distribution_channel <chr>, is_repeated_guest <dbl>,
## #   previous_cancellations <dbl>, previous_bookings_not_canceled <dbl>, ...
```

Insight

The highest lead time for a hotel booking in this data set is 737 days. After using the `arrange()` function to sort the `hotel_bookings` by lead time in descending order, we will notice that 737 is the first row. That is over two years in advance!

Notice that when we just run `arrange()` without saving our data to a new data frame, it does not alter the existing data frame. Check it out by running `head()` again to find out if the highest lead times are first:

```
head(hotel_bookings)
```

```
## # A tibble: 6 × 32
##   hotel          is_canceled lead_time arrival_date_year arrival_date_month
##   <chr>          <dbl>      <dbl>          <dbl> <chr>
## 1 Resort Hotel      0        342            2015 July
## 2 Resort Hotel      0        737            2015 July
## 3 Resort Hotel      0         7            2015 July
## 4 Resort Hotel      0        13            2015 July
## 5 Resort Hotel      0        14            2015 July
## 6 Resort Hotel      0        14            2015 July
## # i 27 more variables: arrival_date_week_number <dbl>,
## #   arrival_date_day_of_month <dbl>, stays_in_weekend_nights <dbl>,
## #   stays_in_week_nights <dbl>, adults <dbl>, children <dbl>, babies <dbl>,
## #   meal <chr>, country <chr>, market_segment <chr>,
## #   distribution_channel <chr>, is_repeated_guest <dbl>,
## #   previous_cancellations <dbl>, previous_bookings_not_canceled <dbl>,
## #   reserved_room_type <chr>, assigned_room_type <chr>, ...
```


If we wanted to create a new data frame that had those changes saved, we would use the `<-` as written in the code chunk below to store the arranged data in a data frame named 'hotel_bookings_v2'

```
hotel_bookings_v2 <-  
  arrange(hotel_bookings, desc(lead_time))
```

Check out the new data frame:

```
head(hotel_bookings_v2)
```

```
## # A tibble: 6 × 32  
##   hotel          is_canceled lead_time arrival_date_year arrival_date_month  
##   <chr>          <dbl>      <dbl>          <dbl> <chr>  
## 1 Resort Hotel      0        737            2015 July  
## 2 Resort Hotel      0        709            2016 February  
## 3 City Hotel        1        629            2017 March  
## 4 City Hotel        1        629            2017 March  
## 5 City Hotel        1        629            2017 March  
## 6 City Hotel        1        629            2017 March  
## # i 27 more variables: arrival_date_week_number <dbl>,  
## #   arrival_date_day_of_month <dbl>, stays_in_weekend_nights <dbl>,  
## #   stays_in_week_nights <dbl>, adults <dbl>, children <dbl>, babies <dbl>,  
## #   meal <chr>, country <chr>, market_segment <chr>,  
## #   distribution_channel <chr>, is_repeated_guest <dbl>,  
## #   previous_cancellations <dbl>, previous_bookings_not_canceled <dbl>,  
## #   reserved_room_type <chr>, assigned_room_type <chr>, ...
```

We can also find out the maximum and minimum lead times without sorting the whole data set using the `arrange()` function. Let's use the `max()` and `min()` functions:

```
max(hotel_bookings$lead_time)
```

```
## [1] 737
```

```
min(hotel_bookings$lead_time)
```

```
## [1] 0
```

In this case, we need to specify which data set and which column using the `$` symbol between their names.

Let's say we just want to know what the average lead time for booking is because our boss asks we how early we should run promotions for hotel rooms. we can use the `mean()` function to answer that question:

```
mean(hotel_bookings$lead_time)
```

```
## [1] 104.0114
```

We should get the same answer even if we use the v2 data set that included the `arrange()` function.

```
mean(hotel_bookings_v2$lead_time)
```

```
## [1] 104.0114
```

Insight

The average lead time is 104.0114 days.

We were able to report to our boss what the average lead time before booking is, but now they want to know what the average lead time before booking is for just city hotels. They want to focus the promotion they're running by targeting major cities.

We know that our first step will be creating a new data set that only contains data about city hotels. We can do that using the `filter()` function, and name our new data frame 'hotel_bookings_city':

```
hotel_bookings_city <-  
  filter(hotel_bookings, hotel_bookings$hotel=="City Hotel")
```

Check out our new data set:

```
head(hotel_bookings_city)
```

```
## # A tibble: 6 × 32  
##   hotel      is_canceled lead_time arrival_date_year arrival_date_month  
##   <chr>      <dbl>      <dbl>          <dbl> <chr>  
## 1 City Hotel      0        6            2015 July  
## 2 City Hotel      1       88            2015 July  
## 3 City Hotel      1       65            2015 July  
## 4 City Hotel      1       92            2015 July  
## 5 City Hotel      1      100            2015 July  
## 6 City Hotel      1       79            2015 July  
## # i 27 more variables: arrival_date_week_number <dbl>,  
## #   arrival_date_day_of_month <dbl>, stays_in_weekend_nights <dbl>,  
## #   stays_in_week_nights <dbl>, adults <dbl>, children <dbl>, babies <dbl>,  
## #   meal <chr>, country <chr>, market_segment <chr>,  
## #   distribution_channel <chr>, is_repeated_guest <dbl>,  
## #   previous_cancellations <dbl>, previous_bookings_not_canceled <dbl>,  
## #   reserved_room_type <chr>, assigned_room_type <chr>, ...
```

We quickly check what the average lead time for this set of hotels is, just like we did for all of hotels before:

```
mean(hotel_bookings_city$lead_time)
```

```
## [1] 109.7357
```

Now, our boss wants to know a lot more information about city hotels, including the maximum and minimum lead time. They are also interested in how they are different from resort hotels. We don't want to run each line of code over and over again, so we decide to use the `group_by()` and `summarize()` functions. we can also use the pipe

operator to make our code easier to follow. we will store the new data set in a data frame named 'hotel_summary':

```
hotel_summary <-  
  hotel_bookings %>%  
  group_by(hotel) %>%  
  summarise(average_lead_time=mean(lead_time),  
            min_lead_time=min(lead_time),  
            max_lead_time=max(lead_time))
```

Check out our new data set using head() again:

```
head(hotel_summary)
```

```
## # A tibble: 2 × 4  
##   hotel          average_lead_time min_lead_time max_lead_time  
##   <chr>              <dbl>          <dbl>          <dbl>  
## 1 City Hotel          110.              0             629  
## 2 Resort Hotel        92.7              0             737
```

Activity Wrap Up

We used functions to change our data, such as `arrange()` , `group_by()` , and `filter()` . We also had some experience using statistical summaries to make insights into our data.